

Drydock Release Process

This is the maintainer runbook for cutting a Drydock release: how to package and publish `drydock-sdd` to PyPI, plus the one-off steps required to launch a **major** release correctly — where the canonical specification and white paper get hosted, and how and where to announce.

Two audiences, two documents:

- End users install with [USER_INSTALLATION.md](#).
- Maintainers (you) release with **this** file.

0. Facts

Item	Value
PyPI distribution	<code>drydock-sdd</code>
Installed command	<code>drydock</code>
Import package	<code>drydock</code>
Version source of truth	<code>src/drydock/__init__.py</code> → <code>__version__</code>
Git remote	<code>git@github.com:webcloudstudio/Drydock.git</code> (branch <code>main</code>)
Versioning	Semantic Versioning; <code>0.x</code> = unstable command surface
Publish script	<code>bin/publish_pypi.sh</code> (production PyPI)
TestPyPI dry-run script	<code>bin/test_publish_pypi.sh</code>
Wheel integrity check	<code>scripts/check_wheel_rigging.py</code>
Video compression	<code>bin/ffmpeg.sh</code>

Credentials live in `~/.pypirc` (never committed) with `[pypi]` and `[testpypi]` sections, each `username = __token__` and a scoped API token. Both publish scripts read the token from `~/.pypirc` (or `PYPI_TOKEN` / `TESTPYPI_TOKEN` in the environment) and require two-factor authentication to be enabled on the account. See [PYPI_NAME_RESERVATION.md](#) for the token/account setup.

Part A — Every release (packaging and publishing)

A1. Pre-flight verification

Run the full gate from a clean tree. Nothing ships red.

```
python -m pytest
ruff check src/ tests/
ruff format --check src/ tests/
```

Confirm the working tree is clean except the intended release changes:

```
git status
```

A2. Bump the version

Edit the single source of truth:

```
# src/drydock/__init__.py
__version__ = "X.Y.Z"
```

Choose the bump per SemVer. While in `0.x`, a breaking command-surface or Typed Specification change is a **minor** bump; additive features and fixes are **patch**.

A3. Update the changelog

Move `## [Unreleased]` items into a new `## [X.Y.Z] - YYYY-MM-DD` section in `CHANGELOG.md`, then reset `[Unreleased]` to "No unreleased changes." Keep the Keep-a-Changelog structure (Added / Changed / Fixed / Removed).

A4. Commit

```
git add -A
git commit -m "Release X.Y.Z"
```

The tag is created by the publish script (A6), not here. Do not create the tag manually — `bin/publish_pypi.sh` refuses to publish if the tag already exists.

A5. Dry-run to TestPyPI (recommended for major releases)

```
bash bin/test_publish_pypi.sh
```

This builds the wheel and sdist, lists the wheel contents, runs `scripts/check_wheel_rigging.py`, and uploads to TestPyPI. Then install from TestPyPI into a throwaway environment and smoke-test the artifact a user actually receives:

```
uv tool install --index https://test.pypi.org/simple/ \
  --index-strategy unsafe-best-match drydock-sdd
drydock --version
drydock config show
drydock init Example      # against a temp $PROJECTS
```

A6. Publish to production PyPI

```
bash bin/publish_pypi.sh
```

The script, in order:

1. Reads the version from `src/drydock/__init__.py` and derives tag `vX.Y.Z`.
2. Aborts if `vX.Y.Z` already exists (forces a version bump for a re-publish).
3. `rm -rf dist/`, then `uv build` (wheel + sdist).
4. Lists wheel contents and runs `scripts/check_wheel_rigging.py` — this asserts the packaged `drydock/resources/Rigging/` is byte-for-byte identical to root `Rigging/` and that the canonical spec is embedded. Publish stops on drift.
5. `uv publish` to PyPI.
6. `git tag -a vX.Y.Z` and pushes `main` and the tag to `origin`.

A7. Post-publish verification

From a clean machine or fresh environment, install the real published artifact:

```
uv tool install drydock-sdd
drydock --version      # matches X.Y.Z
drydock --help
drydock config show
```

The wheel must contain `drydock/`, `drydock/resources/Rigging/`, `drydock/resources/prompts/`, `drydock/resources/QuarterDeck/`, and `drydock/resources/docs/Drydock_Specification.md`. `check_wheel_rigging.py` already enforces this during publish; this step confirms the end-to-end install.

A8. Cut the GitHub Release

```
gh release create vX.Y.Z --title "Drydock X.Y.Z" --notes-file <(sed ...) # or
paste changelog section
```

Attach the rendered spec PDF (`docs/Drydock_Specification.pdf`) and, for a major release, the white paper PDF. Link the PyPI page.

Part B — Major-release one-off steps (host the canonical materials)

Do these once per **major/marketing** launch, not on every patch. The strategy: publish the authoritative materials once, in a few durable, citable homes; make the GitHub repo README the single hub everything else links to. Full plan and rationale in [presentation/distribution.md](#).

B1. Publish the repo and README as the hub

- Repo public at github.com/webcloudstudio/Drydock.
- README leads with the one-idea pitch, the install one-liner, a 60-second example, and links to spec / deck / paper / video.
- Replace any github.com/<your-handle> placeholders (deck Slide 14 and elsewhere).

B2. Host the canonical specification and docs (GitHub Pages)

The canonical spec ships three ways and they must agree:

1. In-repo source of truth: [docs/Drydock_Specification.md](#).
2. Packaged read-only resource inside the wheel.
3. **Rendered public page** — the website you were missing.

Publish the rendered HTML to **GitHub Pages** from [docs/](#) (repo Settings → Pages → source [main](#) / [docs](#)). This gives stable, citable URLs:

```
https://webcloudstudio.github.io/Drydock/                #
docs/index.html – methodology hub
https://webcloudstudio.github.io/Drydock/Drydock_Specification.html # canonical
spec
https://webcloudstudio.github.io/Drydock/presentation/deck.html    # launch deck
```

Regenerate the rendered artifacts before publishing:

```
drydock publish docs/Drydock_Specification.md \
  --output docs/Drydock_Specification.html --theme <theme>
drydock publish docs/Drydock_Specification.md \
  --output docs/Drydock_Specification.pdf --pdf
```

Commit the regenerated HTML/PDF so Pages serves the current release.

B3. Mint a citable DOI for the white paper (Zenodo)

Write the 4–8 page white paper — *the missing process in spec-driven development, and why it's Agile* — export to PDF, and upload to **Zenodo** for a permanent, versioned DOI. Mirror it as HTML on GitHub Pages and link the DOI from the README. Optionally submit the same paper to **SSRN** (quant/financial-engineering framing) and **arXiv** [cs.SE](#) (needs endorsement for a first submission — plan lead time).

B4. Compress and post the video (YouTube)

Compress the raw recording before upload:

```
bash bin/ffmpeg.sh    # docs/presentation/Drydock_Video.mp4 -> *.web.mp4
```

Upload to YouTube **unlisted** first. Title: [Drydock: specification-driven delivery with a real Agile process](#). Add chapters from [docs/presentation/script.md](#) timings, put the install line + repo link + Q&A

in the description and a pinned comment, do an audio/timing pass with 2–3 people, then flip public.

B5. Product Hunt (go-live day)

Prepare a listing: tagline, gallery (deck stills + a 30s clip), and a maker comment. Schedule it for launch day and line up a few people to be present in the comments. It is a one-shot attention spike — don't waste it.

Part C — Announce (drop-a-link venues + sample text)

Post a short, **text-first / link-second** message that points back to the repo (the hub). Rank by signal, not reach. **Stagger over 1–2 weeks** — same-hour cross-posting reads as spam and gets throttled. Be present for the first 48 hours; the launch is the comment thread, not the upload.

C1. Posting order (staggered)

Day	Venue	Link target
1 (Tue–Thu, 8–10am ET)	Hacker News Show HN ; Lobsters (tags practices/ai)	Repo / white paper — not YouTube
2–3	Reddit : r/programming, r/ExperiencedDevs, r/SoftwareEngineering, r/devtools, r/LLMDevs, r/ChatGPTCoding, r/opensource, r/coolgithubprojects	Text post framing the idea; link second. Don't cross-post identically same-day
2–3	Lemmy (programming.dev, lemmy.ml/c/programming)	Same framing as Reddit
Ongoing	GitHub Spec Kit discussions (as a peer — "imports Spec Kit projects"); spec-driven / AI-eng Discords & Slacks (read each self-promo rule); awesome-spec-driven-development PR	Repo
Ongoing	X / Mastodon (fosstodon) / Bluesky thread; LinkedIn (product-owner / process framing)	30s clip, link last
Set-and-forget	AlternativeTo / LibHunt / SaaSHub / Slant / StackShare ; FSF Free Software Directory ; newsletters (Console.dev , TLDR , Pointer.io , The Changelog , Hacker Newsletter)	Repo / write-up

C2. Sample messages

Show HN title:

Show HN: Drydock – governed, spec-driven builds with an Agile process, on your Claude/Codex CLI

Show HN first comment (you, plain text):

Author here. SDD gave us specs but threw away 25 years of software practice for shipping them — there's no process, so you can't reproduce a build. Drydock adds an Agile loop: analyze → plan a dependency graph → build one context-budgeted step at a time → score drift → refit. It runs on the Claude/Codex CLI (no API keys) and imports Spec Kit projects. It's v0 and rough in places (the spec has open [TODO:s](#)). I'd love feedback on the process model and the QuarterDeck review flow. [<repo link>](#)

Short (Reddit / Discord / social):

I built a functional spec-driven-development method that adds the part SDD always skipped — the *process*. It's Agile: you're the Commander (product owner), the LLM is your delivery team, and a dependency-graph build keeps the spec and code in sync so you can actually reproduce a build. Runs on your Claude/Codex subscription, no API keys. Repo + 10-min walkthrough: [<link>](#)

One-liner (directories / awesome-list PR):

Drydock — governed, specification-driven software delivery with an Agile process loop, on your existing Claude/Codex CLI.

C3. Clean-launch rules

1. One write-up is the hub (README or a dev.to post with `canonical_url` set); don't fragment the message.
2. Lead with the problem, not the product.
3. Ship the honest state — "v0, here's what works and what doesn't" beats a glossy reel; the spec's `TODO:s` are a feature to name, not hide.
4. Be present for the first 48 hours — answer every reply fast.
5. Don't blast simultaneously; stagger.
6. Make the repo the destination.

Release checklist (copy per release)**Part A — packaging**

- [] `pytest / ruff check / ruff format --check all green`
- [] `__version__` bumped in `src/drydock/__init__.py`
- [] `CHANGELOG.md` updated (new version section; [Unreleased] reset)
- [] committed "Release X.Y.Z"
- [] TestPyPI dry-run installed and smoke-tested (major releases)
- [] `bin/publish_pypi.sh` succeeded (build + wheel check + publish + tag + push)
- [] `fresh `uv tool install drydock-sdd` smoke-tested`
- [] GitHub Release cut with notes + spec PDF

Part B — major launch (one-off)

- [] README hub finalized; placeholders replaced
- [] GitHub Pages serving current spec HTML + deck
- [] White paper on Zenodo (DOI); mirrored on Pages; SSRN/arXiv if pursuing
- [] Video compressed (`bin/ffmpeg.sh`) and on YouTube (unlisted → public)
- [] Product Hunt listing scheduled

Part C — announce

- [] Sample messages adapted per venue
- [] Posting order staggered on a calendar
- [] Present for first 48h